

MEDIATEK

MT8676 Android GPU User Manual

版本: 1.0
出版日期: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	Lingxiao Wang	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 GPU 4

 1.1 概述 4

 1.1.1 什么是 GPU 4

 1.1.2 为什么需要 GPU 4

 1.1.3 如何使用 GPU 5

 1.2 架构/进程概述 5

 1.2.1 Android 图形系统框架 5

 1.2.2 Arm Mali-G615 架构及功能 6

 1.3 常见问题/故障排除 7

 1.3.1 GPU 渲染分析 7

 1.3.2 GPU 性能分析 9

附件一 附加条款 12

图片目录

图 1-1. Android 图形框架 5

图 1-2. Arm Mali-G615 架构 6

图 1-3. MGD 8

图 1-4. MGD 配置 8

图 1-5. 建立连接 9

图 1-6. Streamline 使用示例 10

表格目录

表 1-1. Arm Mali-G615 feature support 6

1 GPU

1.1 概述

本章节主要介绍 MT8676 GPU 的基本知识。

MT8676 的 GPU 使用的是 Arm Mali-G615，其算力为 1.8T FLOPS。

1.1.1 什么是 GPU

GPU：全称是 Graphics Processing Unit，即图形处理器，又称显示核心、视觉处理器、显示芯片，是一种专门在个人电脑、工作站、游戏机和一些移动设备（如平板电脑、智能手机等）上做图像和图形相关运算工作的微处理器。

1.1.2 为什么需要 GPU

- GPU 作为硬件显卡的“心脏”，地位等同于 CPU 在计算机系统中的作用。同时 GPU 也可以用来作为区分 2D 硬件显卡和 3D 硬件显卡的重要依据。2D 硬件显卡主要通过使用 CPU 来处理特性和 3D 图像，将其称作“软加速”。
- GPU 是并行编程模型和 CPU 的串行编程模型完全不同，导致很多 CPU 上优秀的算法都无法直接映射到 GPU 上，并且 GPU 的结构相当于共享存储式多处理结构，因此在 GPU 上设计的并行程序与 CPU 上的串行程序具有很大的差异。GPU 主要采用立方环境的材质贴图、硬体 T&L、顶点混合、凹凸的映射贴图和纹理压缩、双重纹理四像素 256 位的渲染引擎等重要技术。
- 由于图形渲染任务具有高度的并行性，因此 GPU 可以仅仅通过增加并行处理单元和存储器控制单元便可有效的提高处理能力和存储器带宽。
- GPU 设计目的和 CPU 截然不同，CPU 是设计用来处理通用任务，因此具有复杂的控制单元，而 GPU 主要用来处理计算性强而逻辑性不强的计算任务，GPU 中可利用的处理单元可以更多的作为执行单元。因此，相较于 CPU，GPU 在具备大量重复数据集运算和频繁内存访问等特点的应用场景中具有无可比拟的优势。

1.1.3 如何使用 GPU

- 使用 GPU 有两种方式，一种是开发的应用程序通过通用的图形库接口调用 GPU 设备，另一种是 GPU 自身提供 API 编程接口，应用程序通过 GPU 提供的 API 编程接口直接调用 GPU 设备。
- 通过通用的图形库的方式使用 GPU，都是通过 OpenGL 或 Direct3D 这一类现有的图形函数库，以编写渲染语言（Shading Language）的方法控制 GPU 内部的渲染器（Shader）来完成需要的计算。
- 目前业界公认的图形编程接口主要有 OpenGL 和 DirectX 这两种接口。OpenGL 是当前可用于开发可交互、可移植的 2D 与 3D 图形应用程序的首选环境，也是当前图形应用最广泛的标准。只要在任何一个遵循 OpenGL 标准的环境下都会产生一样的可视化效果。与 OpenGL 类似，DirectX（Directe Xtension）也是一种图形 API。为适应 GPU 应用的需求，DirectX 则根据 GPU 新产品功能的扩充与进展及时地定义新的版本，它所提供的功能几乎与 GPU 提供的功能同步。

1.2 架构/进程概述

1.2.1 Android 图形系统框架

Android 图形系统框架如下所示：

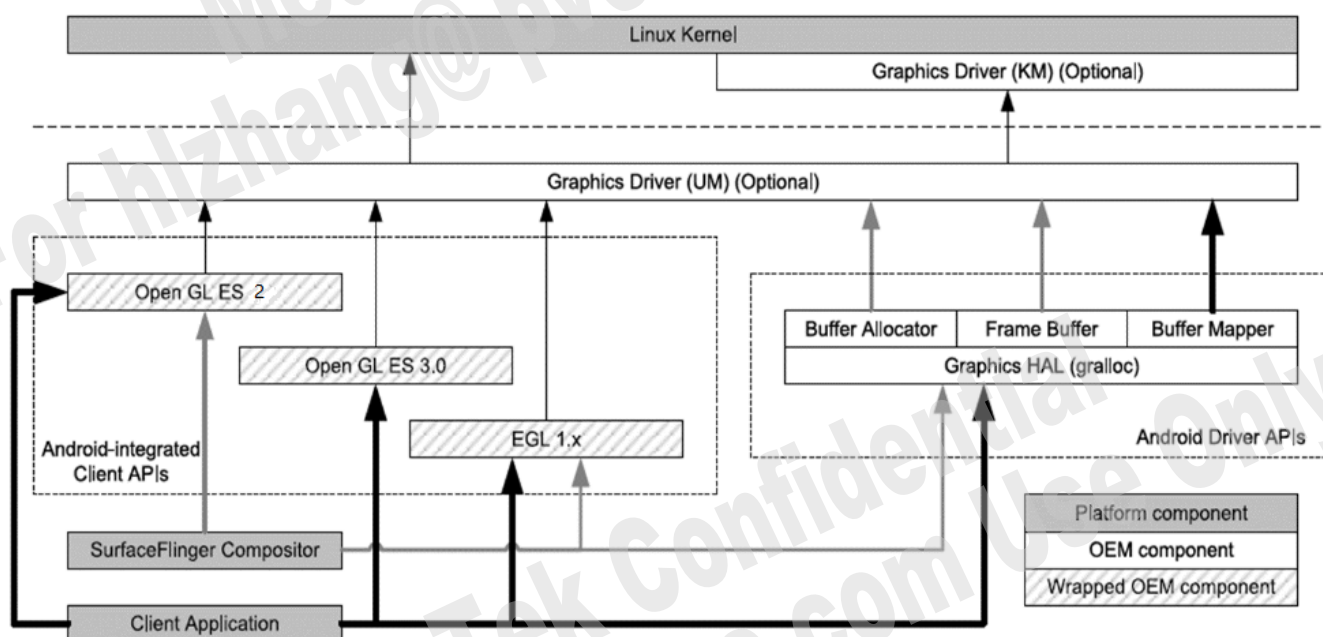


图 1-1. Android 图形框架

1.2.2 Arm Mali-G615 架构及功能

Arm Mali-G615 架构请参考下图：

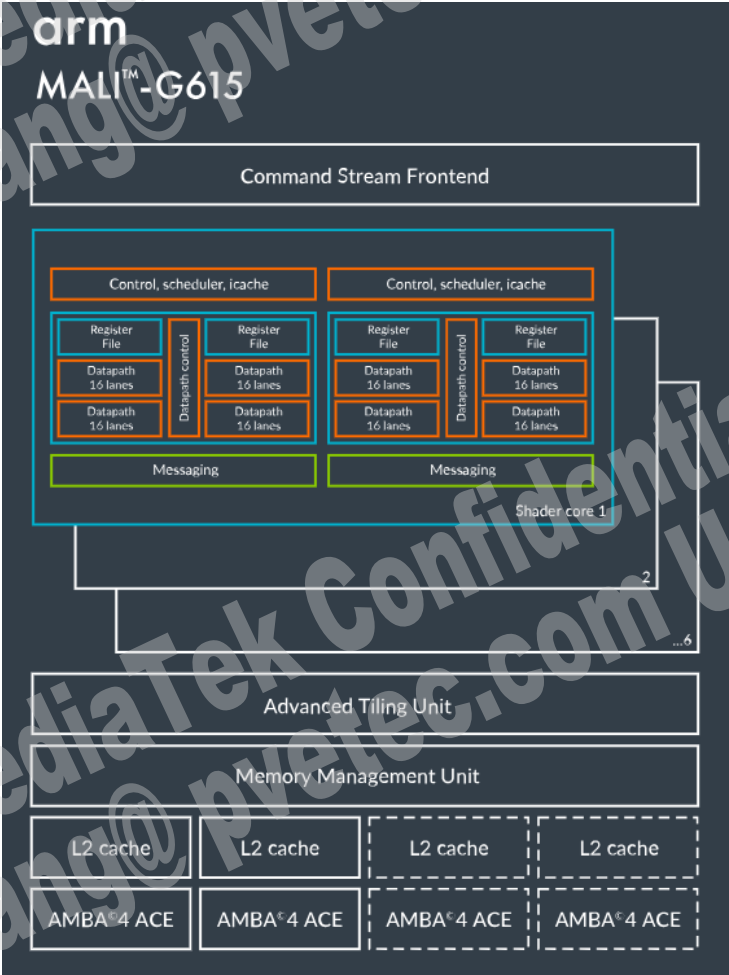


图 1-2. Arm Mali-G615 架构

Arm Mali-G615 Feature Support 请参考表 1-1：

表 1-1. Arm Mali-G615 feature support

Features	Value	Description
Anti-Aliasing	4x MSAA 8x MSAA 16x MSAA	4x Multi-Sampling Anti-Aliasing (MSAA) with minimal performance drop.
API Support	- OpenGL® ES 1.1, 2.0, 3.1, 3.2 - Vulkan 1.1, 1.2, 1.3	Full support for next-generation and legacy 2D/3D graphics applications.

Features	Value	Description
	OpenCL™ 1.1, 1.2, 2.0 Full Profile	
Adaptive Scalable Texture Compression (ASTC)	Low Dynamic Range (LDR) and High Dynamic Range (HDR). Supports both 2D and 3D images.	ASTC offers several advantages over existing texture compression schemes by improving image quality, reducing memory bandwidth and thus energy use.
Arm Frame Buffer Compression (AFBC)	- Version 1.3.2 4x4 pixel block size	AFBC is a lossless image compression format that provides random access to pixel data to a 4x4 pixel block granularity. It is employed to reduce memory bandwidth both internally within the GPU and externally throughout the SoC.
Arm Fixed Rate Compression (AFRC)	- Version 1.0 4x4 pixel block size	AFRC is a lossy image compression format. AFRC can be used for compressing external texture inputs and framebuffer outputs from the GPU. Configurable compression ratio provides guaranteed bandwidth reduction for such surfaces and memory footprint saving.
Variable Rate Shading	- Pipeline, primitive and attachment shading rates - Up to 4x4 shading rate	Variable Rate Shading decouples fragment shading frequency from rasterization frequency, providing the opportunity to make energy savings while maintaining perceived visual quality.

1.3 常见问题/故障排除

1.3.1 GPU 渲染分析

出现屏幕绘制异常时，一般可以从三个方面进行分析，分别是 SF/HWC/Display、GPU 和 APK。判断是否为 SF/HWC/Display 问题，首先可以查看 log 中是否有 display 相关错误，根据 log 进行下一步分析；其次平台有两种叠图方式，可以通过 OVL 或者 GPU 进行叠图，可以通过关闭 HW OVL，强制使用 GPU 进行叠图，查看异常情况；最后可以使用 screenrecord 命令进行录屏，查看录屏结果是否也为渲染异常。如果判断为 SF/HWC/Display 问题可以找相关模块负责人进行下一步分析。

GPU 问题在 log 中搜索是否有 Mali/EGL/GLES/HWUI 等关键字相关的错误，根据错误进行下一步分析。也可以使用一些调试工具，例如 GPUD 和 Graphics API Debugger、Mali Graphics Debugger、RenderDoc 等，这些工具可以帮助分析问题。也可以一些做有关 GPU 的对比实验。

APK 问题需要请 APK 共同分析，是否绘制时使用 GL 接口的问题，或者传入绘制的纹理不对等情况。

1.3.1.1 MGD 使用示例



图 1-3. MGD

Mali Graphics Debugger (MGD) 指 Mali 专用的 debug 工具，可以抓取目标进程的 GPU 的渲染流程进行分析。使用命令对平台需要进行准备操作包括以下命令：

```
adb root;
adb remount ;
adb shell setenforce 0
```

配置 MGD，按照图 1-4 中标红的顺序进行配置。配置完成后需要 set property to MGD，使用以下命令：

```
adb shell "setprop vendor.debug.gpu.provider 'mgd' "
adb shell "getprop vendor.debug.gpu.provider"
adb shell "stop;start"
```

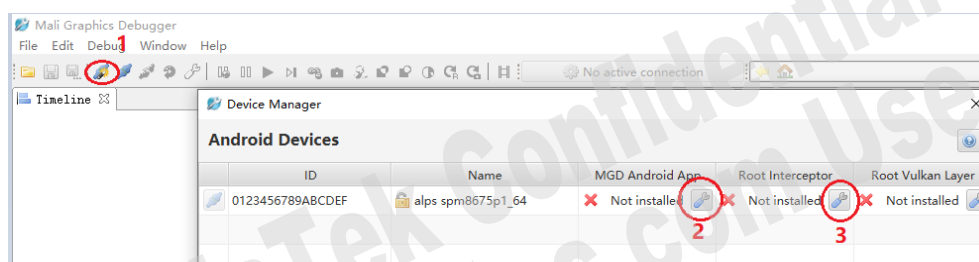


图 1-4. MGD 配置

最后建立平台与 MGD 的连接如图 1-5。连接完成后即可抓取目标进程的渲染过程。

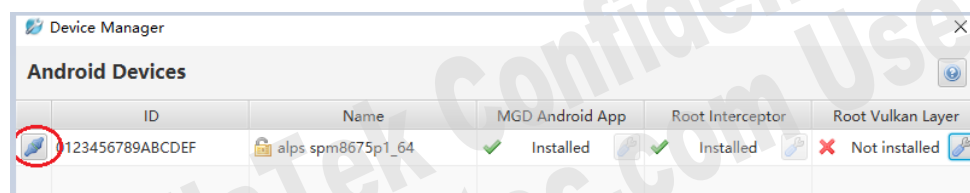


图 1-5. 建立连接

1.3.1.2 GPU 相关对比实验

常见对比实验如下：

- (1) 谷歌对比机是否可复现
- (2) 问题与 Android 系统版本是否相关
- (3) 上一代 GPU 框架（Midgard/Bifrost）平台是否可复现
- (4) 问题是否与 GPU Driver Version 有关
- (5) 关掉 AFBC 是否复现
- (6) 强制 glFinish 是否复现
- (7) 关掉 partial update 是否复现
- (8) 问题是否和 ASTC、MSAA 相关
- (9) HWUI render pipeline 切换实验
- (10) RenderEngine backend 切换实验
- (11) 其他方面等

1.3.2 GPU 性能分析

对于 GPU 的性能问题分析，通常有三个方面包括 GPU 问题、APK 问题和其他模块或系统相关问题。GPU 问题可以查看 Android log 和 kernel log 中有没有 Mali/EGL/GLES/HWUI 关键字的错误 log，根据 log 进行下一步的分析。可以使用工具抓问题场景的 systrace 或 perferro 进行分析，以及使用 ARM Streamline 检查 HW 执行情况，查看具体是哪一个部分影响到 GPU 的性能。也可以做一些针对性的对比实验，细分影响性能的部分。

对于 APK 部分，也可以使用 systrace 或 perferro 进行分析，是否为 APK 原因。其他模块或系统相关问题可以通过 log 和火焰图进行分析。

1.3.2.1 GPU 性能常见对比实验

GPU 性能不达标的常见对比实验如下：

- (1) Fix performance mode 是否达标
- (2) 有无限频因素（如 thermal）
- (3) 是否与 Power Policy 策略有关
- (4) 是否与 Driver Overhead 有关
- (5) 是否与内存带宽/GPU QOS 有关
- (6) 是否与 GPU Driver Version 有关
- (7) Arm Mali Offline Compiler
- (8) 其他方面等

1.3.2.2 Streamline 示例

Streamline 可以查看 GPU HW 的执行情况和执行使用时间。

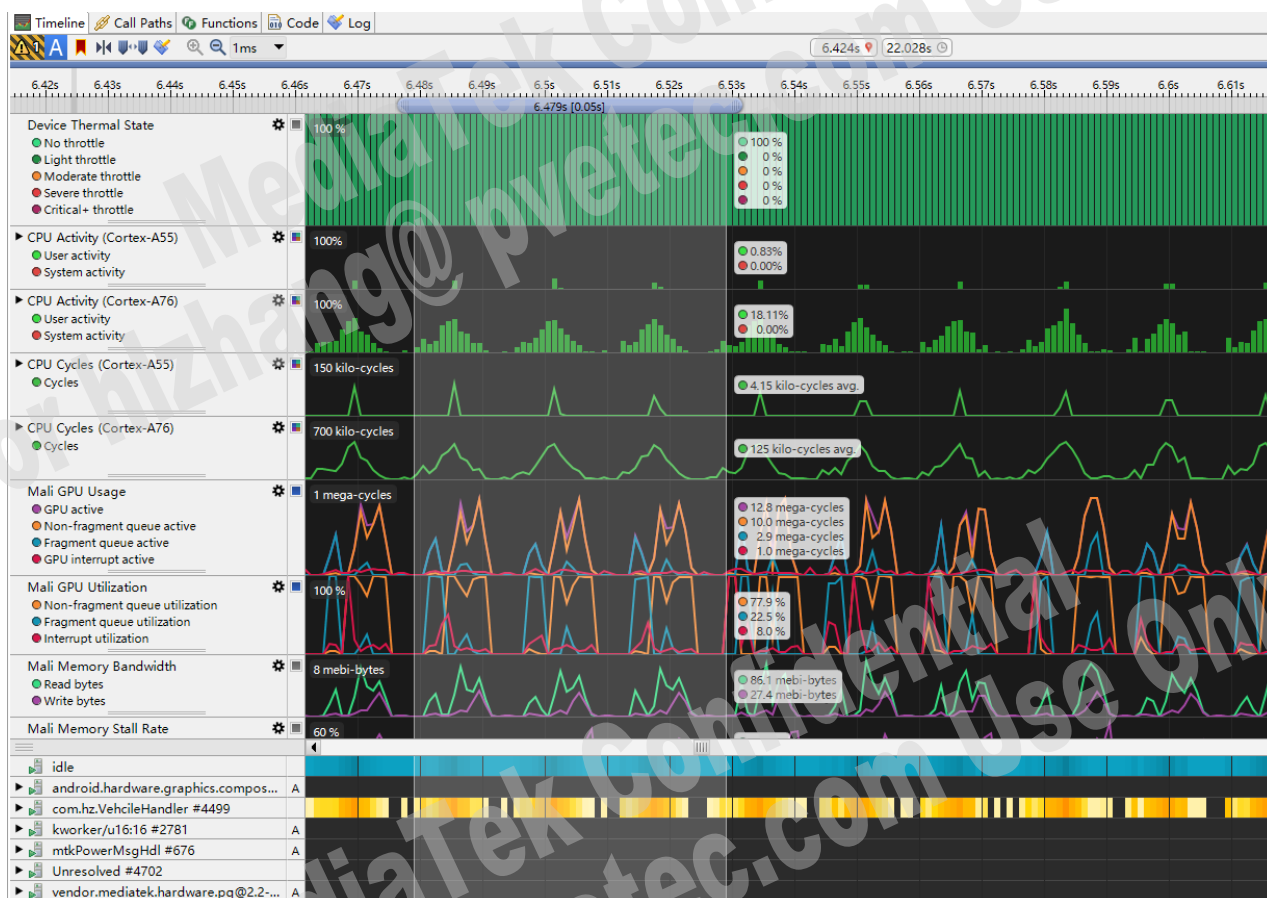


图 1-6. Streamline 使用示例

1.3.2.3 性能优化建议

针对 GPU 的性能优化主要有以下几个建议：

(1) 首先确认性能瓶颈

明确具体是哪一部分影响到平台的性能，具体可以是 CPU、GPU（vertex 或 fragment）、Bandwith 等。确认性能瓶颈后，有针对性地进行优化。

(2) Vertex 负载过重

Vertex 负载过重会造成 GPU 绘制时卡顿，在使用 OpenGL 时需要避免顶点属性资料量过大。

(3) Fragment 负载过重

尽量减少 overdraw，在 shader 中避免使用非必要精度和优化数学表达式等。在 shader 中使用简单有效的语句，可以减少 Fragment 负载。

(4) 带宽瓶颈

针对带宽的优化方向通常为：AFBC、ASTC、render size、mipmapping、pixel format 等。

(5) Driver Overhead

避免使用导致 CPU、GPU 串行运行的某些接口（如 glReadpixels、glFinish），优化每帧 gl 接口的使用数量，提倡使用 VBO、EBO、VAO 等。

(6) 开发者指南（源自 Arm Developer）

Arm GPU Best Practices Developer Guide，

链接：<https://developer.arm.com/documentation/101897/0301?lang=en>。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。